Pré-Requisitos — Configurando o Ambiente de Banco de Dados

Objetivo:
Este é o arquivo que você usará para iniciar os dois bancos de dados ao mesmo tempo.

Instruções:

1. Crie uma pasta nova no seu computador (ex: bootcamp-db).
2. Baixe o arquivo chamado docker-compose.yml que está no material de apoio e coloque dentro dessa pasta.
3. Abra seu terminal nessa mesma pasta.
4. Rode o comando: docker compose up -d
5. Pronto! Seus dois bancos de dados (Postgres e Mongo) estão rodando.

Dados de Conexão:

1. Para o PostgreSQL (DBeaver/Beekeeper/pgAdmin):

   Host: `localhost`

   Porta: `5432`

   Banco de Dados: `orion_db`

   Usuário: `orion_user`

   Senha: `orion_password`

2. Para o MongoDB (MongoDB Compass ou DBeaver):

   Connection String (URI):
   `mongodb://orion_admin:orion_admin_pass@localhost:27017/`

Exercício 1 — Banco relacional (SQL Básico)

Objetivo:
Aprender os comandos básicos de **SQL** para Definição (CREATE TABLE) e Manipulação (INSERT, SELECT).

Descrição:
Você criará as tabelas, definirá suas colunas e chaves, e inserirá alguns dados de teste.

1. Criar a tabela **cursos** que:
    - Tenha as colunas: id (Chave Primária, numérico) e nome_curso (Texto).

2. Criar a tabela **alunos** que:
    - Tenha as colunas: id (Chave Primária), nome (Texto), email (Texto) e curso_id (numérico).
    - Configure curso_id como uma **Chave Estrangeira** (FK) que se relaciona com o id da tabela cursos.

3. Inserir dados:
    - Insira 2-3 cursos na tabela cursos.
    - Insira 3-4 alunos na tabela alunos, relacionando-os com os cursos que você criou.

Critérios de sucesso:

- O comando CREATE TABLE executa sem erros.
- A Chave Estrangeira impede que você insira um aluno com um curso_id que não existe.
- O comando SELECT * FROM alunos; retorna todos os alunos que você inseriu.

Exercício 2 — Banco relacional (JOINs e Filtros)

Objetivo:
Aprender a consultar dados de múltiplas tabelas usando **JOIN** e a filtrar resultados com **WHERE**

Descrição:
Usando o esquema e os dados do Exercício 1, você agora vai responder perguntas complexas que exigem a combinação de dados das tabelas alunos e cursos.

1. Escrever um **SELECT** que:
    - Use INNER JOIN para mostrar o nome do aluno e o nome do curso que ele está fazendo, em uma única consulta.
2. Escrever um **SELECT** que:
    - Use WHERE e JOIN para mostrar apenas os alunos que estão no curso de, por exemplo, "Desenvolvimento Web".
3. Executar um **UPDATE** que:
    - Altere o curso_id de um aluno específico (ex: "Mover Maria para o curso de Ciência de Dados").

Critérios de sucesso:

- A consulta JOIN retorna os nomes corretos (ex: "João Silva", "Desenvolvimento Web").
- A consulta com WHERE filtra corretamente os alunos.
- O UPDATE é bem sucedido e um novo SELECT com JOIN reflete a mudança.

Extras (avançado):

- Escreva um SELECT com LEFT JOIN e WHERE para descobrir quais cursos não possuem nenhum aluno matriculado.

Exercício 3 — Não relacional (CRUD Básico)

Objetivo:
Aprender a estrutura básica de um banco **NoSQL** (MongoDB) e entender o conceito de "Schema Flexível".

Ferramenta Sugerida:
MongoDB Atlas (nuvem gratuita) ou MongoDB (via Docker).

Descrição:
Crie uma coleção (equivalente a uma "tabela") para os posts de um blog. Você irá inserir documentos (equivalente a "linhas") com estruturas diferentes para provar a flexibilidade.

1. Criar uma **coleção**:
    - Nomeie a coleção como posts.
2. Inserir (Create):
    - Insira um documento com: titulo, autor e conteudo.
3. Testar Schema Flexível:
    - Insira um segundo documento com: titulo, autor, conteudo e um campo novo: tags (um array de strings, ex: ["nosql", "flexivel"]).
4. Consultar (Read):
    - Use o comando find() para listar todos os documentos da coleção posts.

Critérios de sucesso:

- find() retorna os dois documentos.
- O segundo documento possui o campo tags e o primeiro não, provando que o banco aceitou estruturas diferentes na mesma coleção.

Exercício 4 — Não relacional (Consultas Avançadas)

Objetivo:
Aprender a consultar (filtrar) dados com base em campos específicos, arrays e documentos aninhados.

Descrição:
Usando a coleção posts do Exercício 3, você vai adicionar dados mais complexos (comentários) e aprender a filtrar os posts com base nesses novos dados.

1. Atualizar (Update):
    - Use updateOne() para adicionar um campo comentarios em um dos seus posts.
    - comentarios deve ser um array de documentos.

2. Criar dois **arquivos Compose**:
    - docker-compose.dev.yml — monta volume local, expõe portas, usa Dockerfile de dev.
    - docker-compose.prod.yml — usa imagem final otimizada, sem montar volumes.

3. Testar ambos os ambientes:
    - Escreva uma consulta find() que retorne apenas posts com a tag "nosql" (consultando dentro de um array).
    - Escreva uma consulta find() que retorne apenas posts onde o autor seja um nome específico.
    - docker compose -f docker-compose.prod.yml up

Critérios de sucesso:

- Ambientes dev/prod funcionam de forma independente.
- O build final segue boas práticas (camadas otimizadas, .dockerignore, imagem leve).
- Estrutura organizada e reutilizável.

O que será avaliado

- **Lógica SQL (Exercícios 1 e 2):**
  - Uso correto de DDL (`CREATE TABLE`).
  - Definição correta de Chaves Primárias (PK) e Chaves Estrangeiras (FK).
  - Uso correto de DML (`INSERT`, `SELECT`, `UPDATE`).
  - Compreensão e aplicação correta de `INNER JOIN` e `WHERE` para filtrar dados.
- **Lógica NoSQL (Exercícios 3 e 4):**
  - Compreensão do Schema Flexível (inserção de documentos com estruturas diferentes).
  - Uso correto de `find()` para consultas.
  - Uso correto de `updateOne()` para adicionar dados (como arrays e documentos aninhados).
  - Habilidade de criar filtros avançados (consultar dentro de arrays e documentos).
- **Ambiente Docker:**
  - Funcionamento correto do `docker-compose.yml` (serviços `db_postgres` e `db_mongo` subindo e acessíveis).
- **Organização do Projeto:**
  - Estrutura de pastas e arquivos conforme solicitado nas instruções de entrega.

Como entregar ?

Os exercícios devem ser entregues em repositório do GitHub através de aprovação de pull-request para a branch main.

A dinâmica fica da seguinte forma:

1. **Criação de um repositório** para entrega dos exercícios (pode ser o mesmo usado para os exercícios de Docker, ex: `exercicios-orion`)
2. **Criação de uma pasta** para este conteúdo para melhor organização: `banco-de-dados/`.
3. **Dentro da pasta `banco-de-dados/`**, crie uma pasta para cada exercício proposto.
   - **Para os Exercícios 1 e 2 (SQL):** Salve seus scripts como arquivos `.sql` (ex: `exercicio-1.sql`, `exercicio-2.sql`).
   - **Para os Exercícios 3 e 4 (NoSQL):** Salve suas consultas em um arquivo de texto simples (ex: `exercicio-3.txt` ou `exercicio-4.js`).
   - **Obs:** Coloque prints dos resultados